```
  const cssClasses = (): string => {
    return `text-center border-b-2 border-transparent ${
      day.filtered === "yes" ? "border-red-700" : ""
    }`
  }

  return (
    <div className={cssClasses()} onClick={calendarClick}>
      {day.day.month} {day.day.date}
    </div>
  )
}

export default SingleDayFilter
```

The context file for schedules creates a useAppDispatch and useAppSelector that are created with the same boilerplate as the ones for the concert page. I reused the names on the theory that no one component is shared between the two stores, but you'd have to give them different names if some component was going to talk to both stores for some reason. Also, I created a subdirectory for the schedule components and also created one for the existing venue components, so all the imports of all those files updated, but I'm not showing all those changes.

The only dynamic elements here are whether the calendar's filter is active, in which case it gets the extra border-red-700 CSS class, and the click handler, which dispatches a calendarToggle event back to the reducer:

appendix_react/01/app/javascript/contexts/schedule_context.ts

```
    case "calendarToggle": {
      const dayInQuestion = state.scheduleDays[action.day]
      const scheduleDays = {
        ...state.scheduleDays,
        [action.day]: {
          ...dayInQuestion,
          filtered: reverseFiltered(dayInQuestion.filtered),
        },
      }
      return {
        ...state,
        scheduleDays: scheduleDays,
```